

NYC 옐로우 택시 End-to-End 분석 — 개인 제출 보고서

Day2 종합실습

제출자 : 임유리

팀원 : 임채현, 임유리, 김광현

데이터 : NYC Yellow Taxi

주제 : 평일 출퇴근 시간대와 비출퇴근 시간대의 택시 요금·이동거리 비교 및 총 요금 예측

코드 : main.py 단일 실행 → scripts/01~11_*.py 순차 파이프라인, report.md / report_v1~v3.md

PART 1

팀 공통 분석 보고서

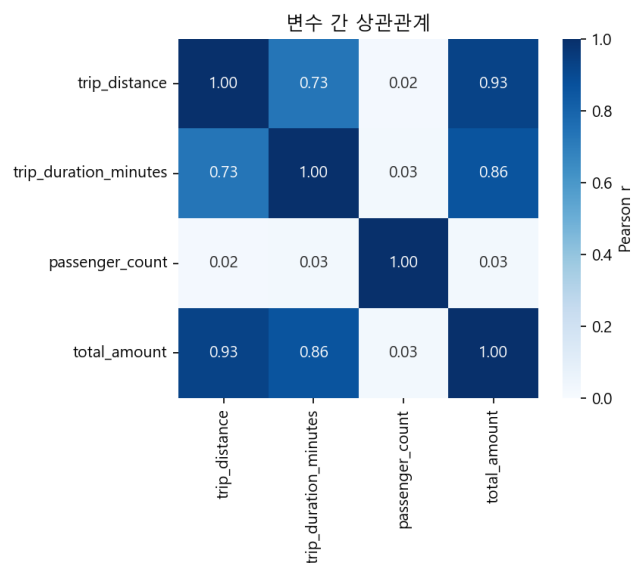
1. 진행 과정 — version1 → version2 → version3

version1: 출퇴근 vs 비출퇴근, 기본 비교

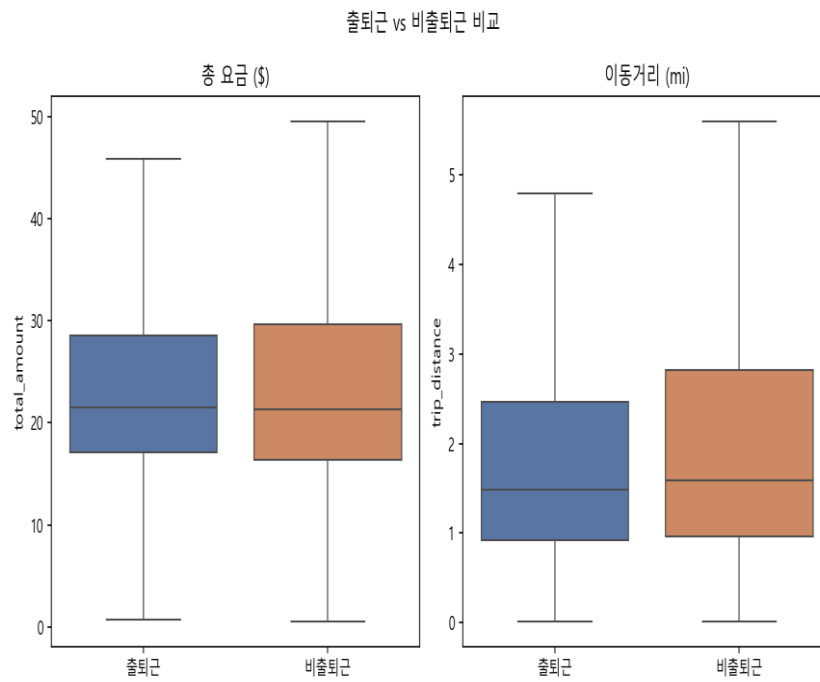
RatecodeID==1·평일 필터링, 이상치 제거 후 906만 건에 대해 기술통계·상관분석·t-test·선형회귀를 수행했다.

```
=== t-test (Welch) ===
      metric  rush_mean  non_rush_mean    diff    t_stat  p_value
total_amount  25.750397    26.366271 -0.615874  -56.457798    0.0
trip_distance   2.332299    2.648771 -0.316473 -158.134712    0.0

=== 선형회귀 (total_amount 예측) ===
RMSE: 4.127  MAE: 2.433  R^2: 0.9335
```



v1 상관관계 (outputs/figures/correlation_heatmap.png)



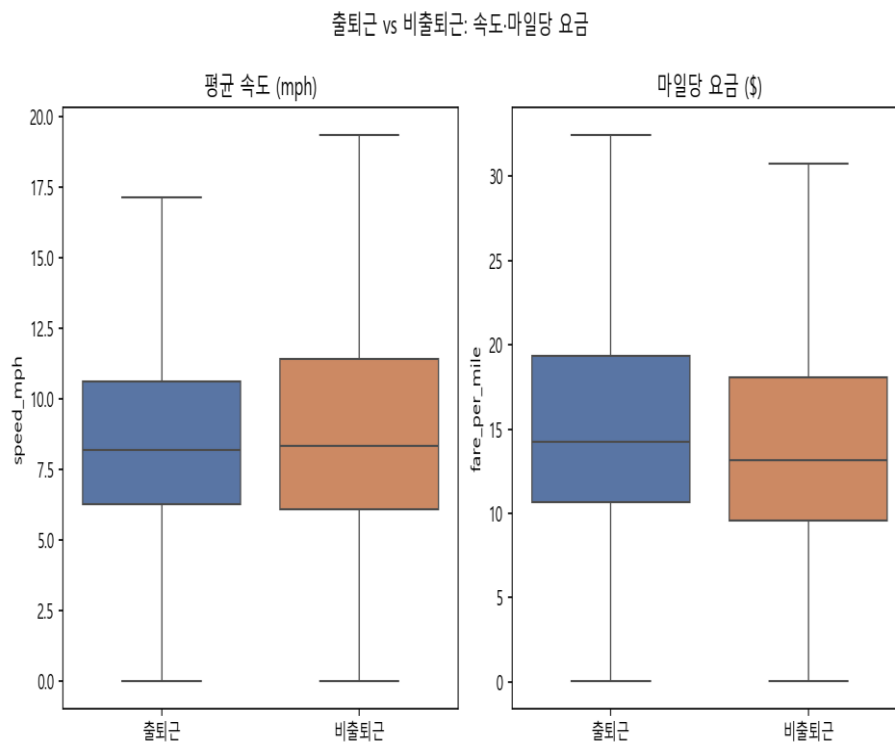
v1 출퇴근 vs 비출퇴근 박스플롯 (outputs/figures/rush_vs_nonrush_boxplot.png)

문제: total_amount 차이가 -2.3%에 불과해, 두 그룹이 “체감상 다를 게 없어 보인다”는 의문이 팀 내에서 제기되었다.

version2: 지표를 쪼개서 재해석 + 효과크기(Cohen's d) 도입

거리가 짧아지는 효과와 속도가 느려지는 효과가 total_amount에서 서로 상쇄된다는 가설을 세우고, speed_mph(평균 속도)·fare_per_mile(마일당 요금)을 파생시켜 재검정했다. 또한 표본이 9백만 건이라 p-value가 항상 0으로 나오는 문제를 보완하기 위해 Cohen's d를 함께 계산했다.

```
=== t-test + Cohen's d (v2) ===
  metric  diff_pct  p_value  cohens_d  effect_size
total_amount    -2.34    0.0   -0.0385   무시가능
trip_distance  -11.95    0.0   -0.1066   무시가능
  speed_mph    -6.17    0.0   -0.1102   무시가능
fare_per_mile    7.25    0.0    0.0453   무시가능
```



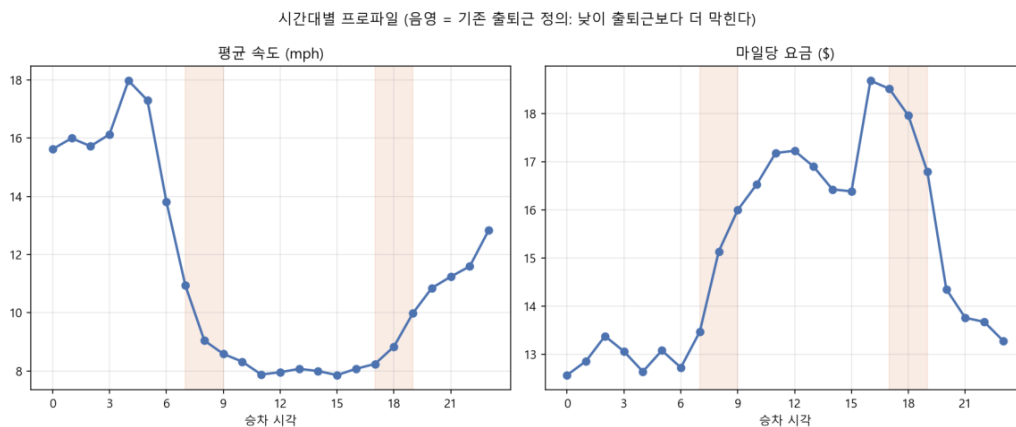
v2 속도·마일당요금 박스플롯 (outputs/figures/speed_fare_boxplot_v2.png)

결론: total_amount 차이가 작아 보인 건 착시가 아니라 사실이었다($d = -0.04$). 즉, “거리는 짧아지고(-12%) 마일당 단가는 오르는(+7%)” 두 효과가 상쇄되고 있었다. 이를 통해, 총액이 아니라 구성을 봐야 한다는 인사이트를 얻었다.

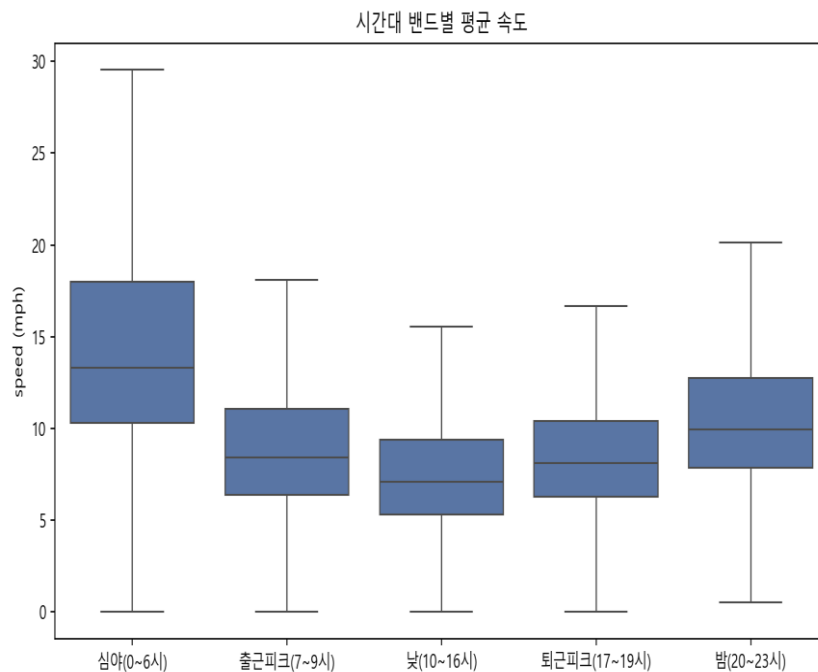
version3: 그룹 정의 재검토

시간대별 평균 속도를 찍어보니 "비출퇴근"으로 묶였던 낮 10~16시(약 8mph)가 출퇴근 피크(8.6~11mph)보다 오히려 더 막히고, 심야 0~6시가 가장 빠르다는 것을 발견했다. 즉 출퇴근/비출퇴근으로 그룹을 나누는 것이, 오히려 효과를 상쇄시키고 있었다. 그렇기 때문에 pickup_hour를 5개 시간대 밴드로 재그룹핑하고 대비가 가장 큰 낮 vs 심야를 헤드라인 검정으로 다시 잡았다.

```
=== t-test: 낮(10~16시) vs 심야(0~6시) + Cohen's d ===
metric  diff_pct      p_value  cohens_d  effect_size
speed_mph    -48.36  0.000000e+00   -1.5253      큰
trip_distance -33.03  0.000000e+00   -0.3893      작음
fare_per_mile  33.31  0.000000e+00    0.1608  무시가능
total_amount  -2.65  1.922217e-139   -0.0420  무시가능
```



v3 시간대별 프로파일 (outputs/figures/hourly_profile_v3.png)



v3 시간대 밴드별 속도 (outputs/figures/band_speed_boxplot_v3.png)

최종 결론: 시간대는 "얼마를 내는가"(total_amount, 어느 그룹 정의를 쓰든 $d \approx -0.04$ 로 거의 무관)가 아니라 "무엇에 대해 내는가"(거리 vs 정체 시간)를 바꾼다. 낮 시간대는 짧은 거리를 비싼 단가로, 심야는 긴 거리를싼 단가로 이동해 총액이 비슷해지는 구조라는 것을 알 수 있었다.

2. 팀원별 개인 의견

임채현

지표 선택 문제를 제기했다. Total_amount 하나만 보고 "차이 없음"으로 결론짓지 않고, 그 지표를 하위 요인으로 쪼개봐야 한다는 문제의식을 제기했고, 이게 version2로 이어졌다.

김광현

그룹 정의 재검토에 대해서 제기했다. 시간대별 속도를 직접 찍어보고 "출퇴근 vs 비출퇴근" 그룹의 한계를 발견했다. Version3에서 시간대별 재설계로 이어졌다.

임유리

처음 결과표를 봤을 때 가장 걸렸던 건 p-value가 v1.v2.v3 전부 0.0 또는 0에 매우 가까운 값으로 찍힌다는 점이었다. 표본이 900만 건을 넘으면 t-statistic이 표본 크기의 제곱근에 비례해서 커지기 때문에, 실제로는 무시해도 되는 차이도 "통계적으로 유의함"으로 나온다는 게 이번 분석에서 그대로 재현됐다. 그래서 p-value 하나만 보고 "유의하니까 의미 있는 차이"라고 결론 내리는 것에 대해서 팀 내에서 계속 생각해보려고 했고, 표본 크기에 영향받지 않는 Cohen's d를 모든 t-test에 같이 계산해서 넣자고 제안했다. 실제로 v2의 4개 지표는 $p < 0.001$ 이면서도 Cohen's d는 전부 '무시가능' 수준이었고, v3에서 그룹을 시간대 밴드로 다시 나누자 speed_mph만 $d = -1.53$ 으로 크다는 결과가 나왔다. 같은 $p\text{-value} < 0.001$ 이라도 그룹을 어떻게 정의하느냐에 따라 효과크기가 이렇게까지 달라질 수 있다는 걸 숫자로 직접 확인한 게 이번 프로젝트에서 가장 크게 배운 부분이다. 데이터 정제 단계에서도 같은 태도로, Pandas-Polars 결과가 shape만 같은 게 아니라 평균·합계까지 실제로 같은지 math.isclose로 검증한 뒤에야 이후 통계 해석에 그 데이터를 신뢰하고 썼다.

3. 팀 종합 의견

세 명의 관점은 서로 다른 지점에서 같은 문제에 대한 의견을 가지고 있었다. 채현님은 "지표 선택"을, 광현님은 "그룹 정의"를, 나는 "검정 방법론"을 문제 삼았다. 셋을 합쳐보면 다음과 같은 순서로 정리된다.

1. 지표를 하나만 보지 말고 구성 요소로 분해한다 (v2)
2. p-value뿐 아니라 효과크기로 실질적 크기를 판단한다 (v2)
3. 애초에 비교 그룹을 데이터에 근거해서 다시 정의한다 (v3)

세 관점 중 어느 하나만 적용했다면 "출퇴근과 비출퇴근은 별 차이 없다"는 얇은 결론에서 멈췄을 것이다. 셋을 순서대로 적용하고 나서야 "총 요금은 시간대와 무관하지만, 거리와 단가의 구성은 시간대에 따라 크게(Cohen's $d = -1.53$) 달라진다"는, 실무적으로 의미 있는 결론에 도달할 수 있었다.

코드 품질 보완 내역

1차 리뷰에서 지적된 4가지 항목을 모두 반영했다.

- **Polars 비교 로딩** : 02_preprocess.py에서 동일 raw parquet을 Pandas·Polars로 각각 로딩해 shape 일치 여부(assert)와 로딩 시간을 비교 출력했다. Polars가 약 3.8배 빨랐다(0.21s vs 0.79s). 따라서 대용량 데이터일수록 차이가 커질 것으로 예상했다.
- **결측치 EDA** : RatecodeID·passenger_count에 각 4,812,280건(전체의 약 25%) 결측치가 있음을 명시적으로 확인했다. RatecodeID=1 필터가 결측치를 자연스럽게 제거하는 구조라 별도 대체(imputation) 없이 진행했다.
- **Plotly 인터랙티브 차트** : 10_plotly_chart.py 추가했다. 시간대별 속도·마일당요금을 hover로 정확한 수치까지 확인 가능한 형태로 outputs/figures/hourly_profile_interactive.html에 저장 했다.
- **sklearn Pipeline + joblib** : StandardScaler+LinearRegression을 Pipeline으로 묶어 학습/평가하고 models/total_amount_regression_pipeline.joblib로 저장했다. 스케일링이 모델과 분리되지 않아 재사용 시 실수 위험이 없어졌다.
- **주석 보강** : 이상치 임계값 근거, Welch's t-test를 쓴 이유(그룹 표본 수 차이), Cohen's d 공식과 해석 기준(Cohen 1988 관례), 시간대 밴드를 다시 나눈 근거를 스크립트 내 인라인 주석으로 추가했다.

4. 결론

- version1: total_amount만으로는 출퇴근 효과를 포착하지 못함을 확인했다.
- version2: 거리·속도 효과가 총 요금에서 상쇄됨을 규명, 효과크기 개념을 도입했다.
- version3: 그룹 정의(출퇴근 vs 비출퇴근) 자체가 상쇄의 원인이었음을 밝히고, 시간대 밴드 재정의로 실질적 효과(속도 $d=-1.53$) 확인했다.

핵심 메시지 : 뉴욕 택시 요금은 시간대에 따라 총액은 거의 그대로지만, 그 총액을 구성하는 거리와 단가(정체 비용)는 시간대에 따라 크게 달라진다.

PART 2

개인 분석

담당 역할 · 실행 결과 · 주요 코드 설명 · 결과 해석 · 개선 사항

1. 담당 역할

데이터 정제 결과가 정말로 믿을 만한지 검증하는 것과, 분석이 끝난 뒤 결과를 report.md로 자동 정리하는 것 두 가지를 맡았다. scripts/02_preprocess.py에서는 Pandas와 Polars로 각각 정제한 결과가 shape·컬럼·결측치·자료형·수치 요약까지 실제로 같은지 비교하는 로직을 만들었고, templates/report.md.j2와 scripts/11_report_jinja.py로 분석 산출물을 모아 report.md를 자동 생성하도록 했다. 결측치·완전 중복 처리를 명시적으로 남기고, 운영체제별 한글 폰트 설정 (scripts/plot_config.py)과 README·스크립트 주석 보강도 같이 진행했다.

결과를 해석할 때는 표본이 900만 건을 넘어 t-test의 p-value가 거의 항상 0에 가깝게 나온다는 점을 이야기 했다. 그리고 p-value 하나만 보고 판단하기보다 Cohen's d처럼 표본 크기에 영향받지 않는 효과크기를 같이 보자고 제안했다.

2. 실행 결과

2.1 실제 실행 화면

```
(.venv) PS C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master> python C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\main.py

=== scripts/01_download_data.py ===
skip (exists): yellow_tripdata_2026-01.parquet
skip (exists): yellow_tripdata_2026-02.parquet
skip (exists): yellow_tripdata_2026-03.parquet
skip (exists): yellow_tripdata_2026-04.parquet
skip (exists): yellow_tripdata_2026-05.parquet

=== scripts/02_preprocess.py ===
pandas load: 0.37s, shape=(18999282, 6)
polars load: 0.17s, shape=(18999282, 6)

결측치 개수 (pandas):
tpep_pickup_datetime      0
tpep_dropoff_datetime     0
trip_distance             0
total_amount             0
RatecodeID              4812280
passenger_count          4812280
dtype: int64

결측치 개수 (polars):
tpep_pickup_datetime: 0
tpep_dropoff_datetime: 0
trip_distance: 0
total_amount: 0
RatecodeID: 4812280
passenger_count: 4812280

raw rows: 18,999,282
after ratecode+weekday filter: 9,368,324
after outlier removal: 9,068,562
rows removed for missing values: 0
duplicate rows removed: 5
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\data\processed\trips_clean.csv (9,068,557 rows)
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\data\processed\trips_clean.parquet (9,068,557 rows)
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\outputs\tables\pandas_polars_comparison.json
analysis_group
비출퇴근      6007171
출퇴근      3861386
Name: count, dtype: int64

=== scripts/03_analysis.py ===
=== 기술통계 (그룹별) ===
      trip_distance      ... trip_duration_minutes
count      mean      std      min      ...      25%      50%      75%
5%      max
analysis_group
비출퇴근      6007171.0      2.648772      3.090577      0.01      ...      7.85      12.716667      20.
000000      299.633333
출퇴근      3861386.0      2.332381      2.719113      0.01      ...      7.45      11.900000      18.5
33333      297.433333

[2 rows x 24 columns]

=== 상관행렬 ===
      trip_distance      trip_duration_minutes      passenger_count      total_amount
trip_distance      1.000000      0.732081      0.021607      0.926025
trip_duration_minutes      0.722081      1.000000      0.026438      0.862545
passenger_count      0.021607      0.026438      1.000000      0.025196
total_amount      0.926025      0.862545      0.025196      1.000000
```

그림 1. 01_download_data.py ~ 02_preprocess.py 실행 로그 — 원천 18,999,282행, 정제 후 9,068,557행, 중복 5건 제거

```

=== t-test (Welch) ===
      metric rush_mean non_rush_mean diff t_stat p_value significant_at_0.05
total_amount 25.750410 26.366273 -0.615863 -56.456730 0.0 True
trip_distance 2.332301 2.648772 -0.316471 -158.133704 0.0 True

저장 경로: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\tables / C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\figures

=== scripts/04_model.py ===
=== 선형회귀 결과 (total_amount 예측, Pipeline: StandardScaler + LinearRegression) ===
      feature coefficient_scaled
trip_distance 10.183224
trip_duration_minutes 6.389679
pickup_hour 0.553033
is_rush_hour 0.519273
intercept 26.160960

RMSE: 4.132
MAE: 2.430
R^2: 0.9333

model saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\models\total_amount_regression_pipeline.joblib

=== scripts/05_report.py ===
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\report_v1.md

=== scripts/06_analysis_v2.py ===
=== 그룹별 기술통계 (v2) ===
      total_amount_mean total_amount_std ... fare_per_mile_mean fare_per_mile_std
analysis_group
비출퇴근 26.367 16.409 ... 15.698 25.204
출퇴근 25.751 15.018 ... 16.837 24.998

[2 rows x 8 columns]

=== t-test + Cohen's d (v2) ===
      metric rush_mean non_rush_mean diff_pct t_stat p_value cohens_d effect_size
total_amount 25.750896 26.367223 -2.337471 -56.499135 0.0 -0.038486 무시가능
trip_distance 2.332020 2.648461 -11.948103 -158.201573 0.0 -0.106588 무시가능
speed_mph 9.113709 9.713144 -6.171377 -169.696818 0.0 -0.110221 무시가능
fare_per_mile 16.837265 15.698385 7.254760 64.694984 0.0 0.045311 무시가능

저장 경로: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\tables / C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\figures

=== scripts/07_report_v2.py ===
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\report_v2.md

=== scripts/08_analysis_v3.py ===
=== 시간대 밴드별 기술통계 (v3) ===
      n speed_mph_mean ... total_amount_mean total_amount_std
time_band
심야(0-6시) 455671 15.532 ... 26.924 18.200
출근피크(7-9시) 1070422 9.345 ... 23.806 15.528
낮(10-16시) 3601867 8.020 ... 26.210 16.823
퇴근피크(17-19시) 1990727 8.909 ... 26.796 14.630
밤(20-23시) 1949861 11.480 ... 26.528 15.440

[5 rows x 9 columns]

```

그림 2. 03_analysis.py ~ 09_report_v3.py 실행 로그 — t-test, Cohen's d, 회귀모델 RMSE/MAE/R² 출력

```

=== t-test: 낮(10-16시) vs 심야(0-6시) + Cohen's d ===
      metric day_mean night_mean diff_pct t_stat p_value cohens_d effect_size
speed_mph 0.020130 15.532163 -40.364371 -624.418704 0.000000e+00 -1.525337 큰
fare_per_mile 17.072709 12.806726 33.310485 89.503801 0.000000e+00 0.160845 무시가능
trip_distance 2.386195 3.563198 -33.032201 -195.841861 0.000000e+00 -0.389301 작음
total_amount 26.209939 26.923573 -2.650590 -25.144788 1.927895e-139 -0.042021 무시가능

저장 경로: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\tables / C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\figures

=== scripts/09_report_v3.py ===
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\report_v3.md

=== scripts/10_plotly_chart.py ===
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\outputs\figures\hourly_profile_interactive.html

=== scripts/11_report_jinja.py ===
saved: C:\Users\dbf15\Downloads\day2-e2e-data-analysis-master\day2-e2e-data-analysis-master\report.md

```

그림 3. 낮 vs 심야 t-test 결과 및 10_plotly_chart.py, 11_report_jinja.py까지 정상 완료

2.2 데이터 정제 및 교차 검증

항목	값
원천 로딩 데이터	18,999,282행 · 6열 (Pandas-Polars 동일)
정제 후 데이터	9,068,557행 · 12열
정제 후 결측치	0건
제거된 완전 중복 행	5건
분석 그룹	출퇴근 3,061,386건 / 비출퇴근 6,007,171건

Pandas-Polars 정제 결과 비교 (outputs/tables/pandas_polars_comparison.json)

비교 항목	결과
원천 데이터 shape	Pass
정제 후 shape	Pass
정제 후 컬럼	Pass
정제 후 결측치	Pass
정제 후 논리 자료형	Pass
주요 수치 컬럼 평균·합계	Pass

전체 정제 결과 비교 : Pass — 두 라이브러리로 각각 정제한 데이터가 구조·값 모두 일치함을 확인했습니다.

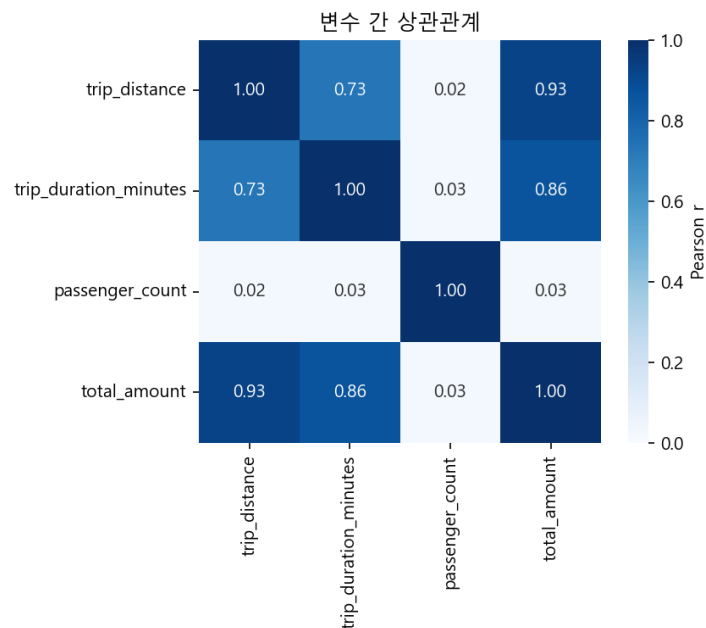


그림 1. 주요 수치 변수 상관관계 히트맵 (outputs/figures/correlation_heatmap.png)

2.3 통계 검정 헤드라인 (v3 — 낮 vs 심야)

지표	낮(10~16시)	심야(0~6시)	변화율	Cohen's d	효과크기
speed_mph	8.020	15.532	-48.36%	-1.525	큼
trip_distance	2.386	3.563	-33.03%	-0.389	작음
fare_per_mile	17.073	12.807	+33.31%	0.161	무시가능
total_amount	26.210	26.924	-2.65%	-0.042	무시가능

시간대별 프로파일 (음영 = 기존 출퇴근 정의: 낮이 출퇴근보다 더 막힌다)

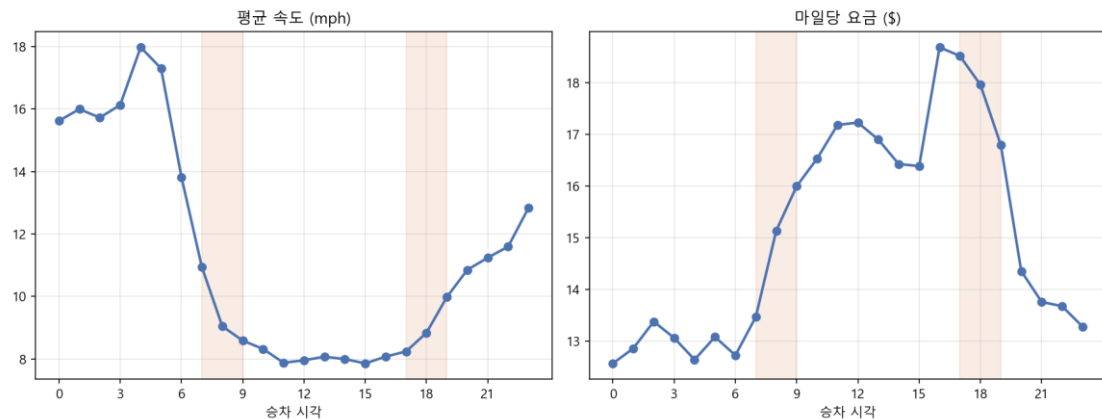


그림 2. 시간대별 속도·마일당 요금 프로파일 (outputs/figures/hourly_profile_v3.png)

2.4 회귀 모델 성능

RMSE	MAE	R ²	평가 표본 수
4.132	2.430	0.9333	1,813,712

StandardScaler + LinearRegression Pipeline, 학습 80% / 평가 20%(random_state=42)로 models/total_amount_regression_pipeline.joblib에 저장됨을 확인했다.

예측 대상인 total_amount가 연속형 수 치이므로 회귀 문제라고 판단했다. 그래서 분류 지표인 정확도 / F1이 아니라 회귀 평가 지표인 RMSE/MAE/R²을 사용했다.

2.5 보고서 자동 생성

python main.py 실행 마지막 단계에서 scripts/11_report_jinja.py가 위 산출물을 모두 취합해 report.md를 자동 생성했다.

3. 주요 코드 설명

3.1 Pandas·Polars 교차 검증 — scripts/02_preprocess.py

같은 원천 Parquet을 Pandas와 Polars로 각각 로딩한 뒤, 동일한 필터·결측치·중복 처리·파생 컬럼 로직을 두 라이브러리에 그대로 적용하고 결과를 비교했다. 핵심은 자료형처럼 두 라이브러리의 표현 방식이 다른 값을 비교 가능한 공통 형태로 정규화하는 부분이라고 볼 수 있다.

```
def normalize_pandas_dtype(dtype):
    if pd.api.types.is_datetime64_any_dtype(dtype):
        return "datetime"
    if pd.api.types.is_integer_dtype(dtype):
        return "integer"
    ...

# Pandas 정제: 기본 요금제·평일만 남기고 이상치·결측치·완전중복 제거
df = df[df["RatecodeID"] == 1]
df = df[df["is_weekday"]]
df = df[(df["trip_duration_minutes"] > 0) & (df["trip_duration_minutes"] < 300)
        & (df["trip_distance"] > 0) & (df["trip_distance"] < 100)
        & (df["total_amount"] > 0) & (df["total_amount"] < 500)]
df = df.dropna(subset=COLS).copy()
df = df.drop_duplicates().copy()

# Polars도 동일한 조건을 메서드 체이닝으로 재현
df_pl_clean = (df_pl.filter(pl.col("RatecodeID") == 1)
    ...
    .drop_nulls(COLS)
    .unique(maintain_order=True)
    .select(list(df.columns)))
```

마지막으로 두 결과의 shape, 컬럼, 결측치 수, 정규화한 자료형, 그리고 주요 수치 컬럼의 평균·합계를 math.isclose로 비교해 outputs/tables/pandas_polars_comparison.json에 저장했다. shape만 비교하면 우연히 행/열 수가 같아도 실제 값이 다를 수 있어, 평균·합계까지 비교 범위를 넓혔다.

3.2 Jinja2 최종 보고서 자동화 — scripts/11_report_jinja.py, templates/report.md.j2

분석 산출물(비교 JSON, 기술통계·상관·t-test CSV, 회귀 평가 결과)을 읽어본 뒤 보고서에 필요한 수치만 가공하고, 표현 형식은 템플릿(report.md.j2)에 위임하는 구조로 작성했다. 즉 스크립트는 “무엇을 보여줄지”, 템플릿은 “어떻게 보여줄지”를 담당하도록 역할을 나눴다.

```
comparison_checks = [
    {"label": "원천 데이터 shape", "passed": comparison["same_raw_shape"]},
    {"label": "정제 후 shape", "passed": comparison["same_cleaned_shape"]},
    ...
]

environment = Environment(loader=FileSystemLoader(TEMPLATES),
                          autoescape=False, trim_blocks=True, lstrip_blocks=True)
environment.filters["comma"] = comma # 천 단위 구분 커스텀 필터
template = environment.get_template("report.md.j2")

rendered = template.render(
    raw_rows=comparison["pandas_raw_shape"][0],
    cleaned_rows=len(df),
    comparison_checks=comparison_checks,
    analysis_stages=analysis_stages, # v1→v2→v3 개선 과정 표
    ...
)
out_path.write_text(rendered, encoding="utf-8")
```

3.3 운영체제별 한글 폰트 설정 — scripts/plot_config.py

```
candidates = {
    "Windows": ("Malgun Gothic", "Noto Sans CJK KR", "NanumGothic"),
    "Darwin": ("AppleGothic", "Noto Sans CJK KR", "NanumGothic"),
    "Linux": ("Noto Sans CJK KR", "NanumGothic", "Malgun Gothic"),
}.get(platform.system(), (...))

available = {font.name for font in font_manager.fontManager.ttflist}
selected = next((name for name in candidates if name in available), "DejaVu Sans")
plt.rcParams["font.family"] = selected
plt.rcParams["axes.unicode_minus"] = False # 한글 폰트에서 음수 기호 깨짐 방지
```

정적 차트를 생성하는 모든 스크립트가 이 함수를 공통으로 호출하도록 해, 실행 환경에 따라 한글 라벨이 깨지는 문제를 줄였다.

4. 결과 해석

제일 걸렸던 건 v1·v2·v3 t-test의 p-value가 전부 0 또는 0에 매우 가까운 값으로 나온다는 점이였다. 표본이 900만 건을 넘으면 t-statistic이 표본 크기의 제곱근에 비례해 커져서, 실제로는 무시해도 되는 차이조차 "통계적으로 유의함($p < 0.001$)"으로 찍혔다. 그래서 이번 분석에서는 p-value 대신 표본 크기에 영향받지 않는 Cohen's d를 기준으로 삼았다.

v1의 출퇴근 vs 비출퇴근 total_amount 차이(-\$0.62)는 $p < 0.001$ 이었지만, v2에서 Cohen's d를 구해보니 -0.04로 사실상 무시 가능한 수준이었다. 그런데 v3에서 그룹을 시간대 밴드로 다시 나누니 낮과 심야의 speed_mph 차이는 $d = -1.53$ 까지 올라갔다. 같은 $p < 0.001$ 이라도 그룹을 어떻게 정의하느냐에 따라 효과크기가 이만큼 달라질 수 있다는 걸 세 버전을 거치며 직접 확인했다.

데이터 정제 검증도 비슷하게 진행했다. Pandas·Polars 결과가 shape만 같다고 실제 값까지 같다는 보장은 없어서, 평균·합계까지 math.isclose로 맞춰본 뒤에야 두 라이브러리 정제 로직이 실제로 동일한 결과를 낸다고 판단했고, 그 데이터를 이후 통계 해석에 썼다.

결론적으로 시간대별로 총요금(total_amount)은 거의 그대로지만($d = -0.04$) 그 요금을 구성하는 이동거리와 속도는 크게 달라진다($d = -1.53, -0.39$). "얼마를 내는가"보다 "무엇에 대해 내는가"가 시간대에 따라 바뀌며, 관찰 데이터라 인과관계까지는 단정할 수 없다는 점도 유의했다.

5. 개선 사항

5.1 Pandas-Polars 교차 검증 항목 확장

기존에는 두 라이브러리로 원천 데이터를 불러온 뒤 shape만 비교했다. Polars에도 Pandas와 동일한 정제 로직을 적용하고 컬럼명·결측치 수·논리 자료형·주요 수치 컬럼의 평균과 합계까지 비교 항목을 넓혔다.

5.2 결측치·중복 처리 명시화 및 저장 형식 이원화

분석 대상 컬럼의 결측치를 명시적으로 제거하고, 완전히 동일한 중복 행을 제거하도록 바꿨다(중복 5건 제거하여 최종 9,068,557행을 사용했다). 정제 데이터는 Parquet뿐 아니라 CSV로도 같이 저장했다.

5.3 Jinja2 기반 최종 보고서 자동 생성

templates/report.md.j2와 scripts/11_report_jinja.py를 새로 만들어, 비교 JSON·통계표·모델 평가 결과를 템플릿에 넘겨 report.md를 자동 생성하게 했다. requirements.txt에 jinja2를 추가하고 main.py 마지막 단계에 연결했다.

5.4 운영체제별 한글 폰트 설정과 README·주석 보강

scripts/plot_config.py를 추가해 Windows(맑은 고딕)·macOS(AppleGothic)·Linux(Noto Sans CJK KR) 순으로 설치된 폰트를 찾고, 없으면 기본 폰트로 대체하도록 했다. 또 프로젝트 목적·실행 방법·평가 항목별 코드 위치를 정리한 README.md를 추가하고, 각 스크립트에 역할·입출력·통계적 판단 근거를 설명하는 주석을 보강했다.

5.5 전처리 변경에 따른 산출물 전체 재생성

중복 제거로 최종 행 수가 바뀐 뒤, 통계표·모델 평가 결과·PNG/Plotly 차트·v1~v3 보고서·Jinja2 최종 보고서를 전부 다시 생성해서 코드와 제출 결과가 어긋나지 않도록 맞췄다.

5.6 앞으로 개선하고 싶은 점

기능 추가와 별개로, 코드 구조 자체는 아직 정리가 필요하다고 본다. 지금은 main.py가 scripts/01~11_*.py를 subprocess로 순서대로 실행하는 방식이라, 각 스크립트가 독립 프로세스로 돌면서 함수를 서로 import해서 재사용할 수 없다. speed_mph·fare_per_mile 파생 로직과 cohens_d 함수가 06_analysis_v2.py와 08_analysis_v3.py에 그대로 중복돼 있는 게 그 결과라고 할 수 있다. 이 둘을 scripts/metrics.py 같은 공통 모듈로 분리하면 로직이 하나로 합쳐지고 수정할 때 두 파일을 동시에 고칠 필요가 없어진다.

테스트 쪽도 아쉬운 부분이 있다. scripts/test_preprocess.py는 02_preprocess.py의 RUSH_HOURS·분류 로직을 그대로 다시 작성해서 검증하는데, 02_preprocess.py 자체에서 함수를 가져와 테스트하는 게 아니라서 원본 코드가 바뀌어도 테스트가 이를 못 잡아낼 수 있다. 전처리 로직을 함수로 빼서 테스트에서 직접 import하는 구조였다면 더 신뢰할 수 있는 회귀 테스트가 됐을 것 같다.

ROOT = Path(__file__).resolve().parent.parent 같은 경로 설정도 스크립트마다 반복된다. plot_config.py 처럼 경로·설정만 모아두는 config.py를 하나 두고 각 스크립트가 거기서 import해 쓰는 구조였다면 스크립트 수가 늘어나도 경로 관리가 더 쉬웠을 것이라고 생각한다.